

```
1  import datetime
2
3  while True
4      comando_utente = input("Cosa vuoi sapere? ")
5      if comando_utente == "esci":
6          print("Arrivederci!")
7          break
8      else:
9          print(assistente_virtuale(comando_utente))
10
11 def assistente_virtuale(comando):
12     if comando == "Qual è la data di oggi?":
13         oggi = datetime.datetime.today()
14         risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
15     elif comando == "Che ore sono?":
16         ora_attuale = datetime.datetime.now().time()
17         risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
18     elif comando == "Come ti chiami?":
19         risposta = "Mi chiamo Assistente Virtuale"
20     else:
21         risposta = "Non ho capito la tua domanda."
22     return risposta
23
```

Esercizio 1) Capire cosa fa il programma senza eseguirlo

Questo è uno script che simula un assistente virtuale da terminale che risponde a determinate domande (in questo caso su data e ora), avendo importato il modulo specifico *datetime*.

Quando l'utente chiede una di queste domande, lo script prende le informazioni dal modulo e le riporta correttamente. Risponde anche alla domanda sul proprio nome, chiamandosi Assistente Virtuale.

Qualora l'utente volesse uscire, è presente l'input "esci", che spezza il ciclo *While* con un *break* e fa terminare il programma.

Nella seconda parte del codice è stata determinata una definizione, inserendo il ciclo *if/elif/else* con le varie opzioni per avere le diverse risposte: al verificarsi di una delle condizioni, si riceve la risposta. Infine viene aggiunta anche l'opzione "Non ho capito la domanda", nell'eventualità l'utente ponga una domanda diversa da quelle previste e rischi di bloccare il programma.

Esercizio 2) Individuare nel codice sorgente le casistiche non standard che il programma non gestisce (esempio, comportamenti potenziali che non sono stati contemplati).

- Una delle prassi più comuni quando si cerca di fare un programma con input, è quella di non usare domande già create. Perché l'utente deve digitare le stesse domande per ottenere le risposte corrette. Qualora sbagliasse anche un piccolo dettaglio (che ore sono o che ora è, ad esempio) il programma darebbe errore.
- Così come il comando "esci" (riga 5) non è formattata Case sensitivity, per cui se l'utente scrivesse "Esci", il programma non termina come dovrebbe.

- Se l'utente inserisce dei numeri anzichè dei caratteri, serve che il programma lo segnali. In questo caso andrà a capo ripetendo "Cosa vuoi sapere?"

Esercizio 3) Individuare eventuali errori di sintassi / logici.

1. Riga 3 - Errore di sintassi; la mancanza dei due punti : dopo *while*.
2. Riga 7 – Il *break* messo così rompe solo il loop, ma non termina davvero il programma.
3. Riga 9 – Il print scritto così stampa il return della funzione. Il *return risposta* è presente solo nel blocco finale con *else*.
4. Riga 11 – Innanzitutto la definizione è determinata dopo il loop principale. Inoltre la funzione che usa la variabile della riga 4, riceve il parametro *comando*, però sono usati nomi diversi rendendo il codice confuso.
5. Riga 14 – Sono presenti dei valori intermedi inutili ai fini del codice. Ad esempio la variabile temporanea *oggi* è inutile, se successivamente si specificano gli oggetti del modulo importato.
6. Riga 17 – Nel formato tra parentesi mancano i secondi, ma soprattutto *ora_attuale* è un oggetto *datetime.time* e non *datetime.datetime*.

Esercizio 4) Proporre una soluzione per ognuno di essi.

1.

```
def assistente_virtuale(comando):
```

 ← funzione messa prima

```
while True:
```

 ← loop dopo la funzione con i due punti

```
    print(assistente_virtuale(...))
```
2.

```
if comando_utente == "esci":
```



```
    print("Arrivederci!")
```



```
    quit()
```

 ← è una funzione built-in di Python, non richiede nessun import aggiuntivo ed è più leggibile e immediata, soprattutto per programmi semplici come questo.
3.

```
risposta = assistente_virtuale(comando_utente)
```



```
if risposta:
```

```
    print(risposta)
```

```
else:
```

```
    print("Errore: nessuna risposta generata.")
```
4.

```
risposta = assistente_virtuale(comando)
```



```
if risposta:
```

```
    print(risposta)
```

```
else:
```

```
    print("Errore: nessuna risposta generata.")
```
5. & 6.

```
risposta = "La data di oggi è " + date.today().strftime("%d/%m/%Y")
```



```
risposta = "L'ora attuale è " + datetime.now().strftime("%H:%M:%S")
```